How NIXNAN works: from load to the first HMMA report

Report accompanying the slides `nixnan-walkthru-tc-rd_nixnan`

Ganesh Gopalakrishnan

2026-10-19

Contents

1 Setup and the environment trap (slide "Setup") **2**

2 Callback architecture **2**

3 Lifecycle **2**

4 First launch: instrumenting a function **2**

5 Compact encoding **3**
- 5.1 `instructions.json` . . . 3
- 5.2 `reginfo` . . . 3
- 5.3 Register extraction . . . 3
- 5.4 Host table . . . 3

6 The two injected calls **3**

7 Device side **4**

8 The channel **4**

9 End-of-run report **4**

10 Part A: an ordinary SASS instruction **5**

11 Part B: HMMA **5**

12 Not covered **5**

Scope and provenance

Everything below comes from reading NIXNAN (branch `print_histo_instrn`, commit `ce35185`): `nixnan.cu`, `inject_funs.cu`, `instruction_info.cu/.cuh`, `instructions.h/.json`, `common.cuh`, `exception_info.cuh`, `fp_utils.cuh`, `recording.cu/.h`, `reginfo.cuh`, and NVBit 1.8's `core/utils/channel.hpp`, plus real runs of `rd_nixnan` (scalar) and `tensor_core/rd_tc` on an RTX 3090, CUDA 12.4, whose raw logs are in `latex/data/`. Statements marked *(computed)* or *(my reading)* were derived by hand and not observed in a run.

1 Setup and the environment trap (slide "Setup")

Run with `env -u LD_LIBRARY_PATH LINE_INFO=1 PRINT_ILL_INSTR=1 TOOL_VERBOSE=1 LOGFILE=... LD_PRELOAD=~/repos/nixnan/nixnan.so <binary>`. With my normal `LD_LIBRARY_PATH` (a nonexistent `/usr/local/cuda-13.2/lib64` and a dyninst directory) NIXNAN printed its banner and no reports; unsetting it fixed that. I did not determine which entry matters. Before this, a driver-module/library mismatch (595.84 vs 595.99.02) made NVBit see no launches at all; a reboot fixed that.

2 Callback architecture

`LD_PRELOAD` puts NIXNAN and the NVBit core in the process. NVBit intercepts CUDA driver calls and invokes four tool callbacks: `nvbit_at_init`, `nvbit_tool_init(ctx)`, `nvbit_at_cuda_event(ctx,is_exit,cbi` and `nvbit_at_ctx_term`. The application is unmodified.

3 Lifecycle

`nvbit_at_init`. Reads all configuration through `GET_VAR_INT/STR` (which also build the banner help text): `INSTR_BEGIN/END`, `TOOL_VERBOSE`, `ENABLE_FUN_DETAIL`, `PRINT_ILL_INSTR`, `SAMPLING`, `INSTRUMENT_EXCEPTIONS`, `INSTR_MEM`, `LOGFILE`, `LINE_INFO`, `LOG_KERNELS`, `MAX_ERRORS`, `TIME_KERNELS`, `FUNCTION_WHITELIST/BLACKLIST` (mutually exclusive), plus the histogram settings. It sets `CUDA_MANAGED_FORCE_DE` because the device channel object is `__managed__`.

`nvbit_tool_init`. Prints "Initializing GPU context…", reads the white/black list, constructs `recorder(TABLE_SIZE)` with `TABLE_SIZE` $= 2^{17}$, sets `recv_thread_started`, initialises `channel_host` with packet size `sizeof(exception_info)` and starts `recv_thread`.

Launch entry. For `cuLaunchKernel`, the `_ptsz` variant and the cooperative variants: set `recv_thread_receivin` call `instrument_function`; print "running kernel [name] …" the first time (always, with `ENABLE_FUN_DETAIL`); increment the per-name counter; call `nvbit_enable_instrumented(ctx,f, sampling>0 ? count%sampling==0 : true)`.

Launch exit. Optional timing, then `cudaDeviceSynchronize()` and a `cudaGetLastError()` check.

`nvbit_at_ctx_term`. Stops and joins the receiver, copies counters back, prints the report (Section 9).

4 First launch: instrumenting a function

`instrument_function` collects `nvbit_get_related_functions` plus the kernel and, for each function not already in `instrumented_functions`, applies `should_instrument` (white/black list, off in `LOG_KERNELS` mode). For each instruction from `nvbit_get_instrs`: `get_reginfo` (skip if no operand has an entry and it is not a memory instruction), possibly histogram instrumentation, then either memory instrumentation or exception instrumentation. Exception instrumentation does `mk_entry` (gets `inst_id`) and two `nvbit_insert_calls`. Because this is done once per function, later launches cost only the toggle.

5 Compact encoding

5.1 instructions.json

Keys are opcode strings, some regular expressions (e.g. the FFMA key allows .FTZ, rounding and .SAT). get_reginfo tries each key with std::regex_match against instr->getOpcode(), falls back to the plain opcode, then reads "registers": one entry per operand with type (f16/bf16/f32/f64), count (values) and optional div0. Opcodes absent from the table are ignored; I verified F2FP is absent.

5.2 reginfo

```
struct reginfo { int count:6; int32_t type:4; bool half_h0:1, half_h1:1, div0:1;
                 unsigned operand:2; int num_regs:4; bool f64high:1; int32_t reserved:12; };
static_assert(sizeof(reginfo)==sizeof(int32_t));
```

The whole descriptor travels as one 32-bit constant argument. count and num_regs are *signed* bit-fields, so num_regs is reliable only to 7 even though get_reginfo asserts get_num_regs<16 (my reading). The operand field is 2 bits: four operands maximum (OPERANDS=4).

5.3 Register extraction

get_regs converts operand k into a list of deferred reginsertion lambdas: for REG/UREG one nvbit_add_call_arg_reg_val (or ureg_val) per register, indices clamped to 255; for CBANK one nvbit_add_call_arg_cbank_val per 4-byte word; double-precision immediates are classified on the host and merely warned about; integer immediates are ignored; generic operands containing NAN/INF text produce a warning. Registers needed: f16/bf16 $\lceil n/2 \rceil$, f32 n, f64 $2n$. f64high opcodes (suffix 64H) start one register lower and use only the high word.

5.4 Host table

recorder::mk_entry stores SASS text, function name (cut at the first < or (), operand types and, with LINE_INFO, the file:line from nvbit_get_line_info under a temporary SIGSEGV handler. The returned index is the inst_id. The device array is indexed (id<<7)|(op<<5)|exce, i.e. 32 flag-combinations $\times$ 4 operands per instruction, uint32 each; at 2^{17} instructions this is 2^{24} words (64 MiB).

6 The two injected calls

AFTER (IPOINT_AFTER): arguments: guard predicate, 64-bit device_recorder (a device pointer), inst_id, 64-bit pointer to channel_dev, arg_count $= 1 +$ #lambdas, the reginfo of operand 0, then operand 0's register values.

BEFORE (IPOINT_BEFORE): same prefix, arg_count $= \sum$ num_regs $+$ (#operands $- 1$), then for each source operand its reginfo followed by its register values.

Reading the destination after and the sources before matters when a register is both (e.g. accumulate in place). The guard predicate argument makes predicated-off threads return immediately.

7 Device side

`nixnan_check_regs` is `extern "C" __noinline__` and variadic. After the predicate test it computes the active mask and first active lane, then iterates `arg_count`: each `reginfo` is followed by `count` values (FP16/BF16 consume two values per 32-bit register; FP64 consumes low then high words). Classification: `half2_classify`, `bf162_classify`, `float_classify`, `double_classify`, each ORing the flags NaN(1), Inf(2), -Inf(4), Sub(8), and Div0(16, zero when requested). Per-operand results are OR-ed across lanes with `__shfl_sync`. The first active lane then, for every operand with a non-zero flag word, atomically increments `device_errors[idx]`; *only if the old value was 0* does it push two `exception_info` packets (`skip=0`, `skip=1`) to the channel.

8 The channel

Device push. `atomicAdd` on the 64-bit head reserves `nbytes`; if the reservation passes `buff_end` the first warp past the end waits until tail equals its pointer (all earlier writers finished) and calls `flush()`, other warps spin on head; otherwise `memcpy` then `atomicAdd` on tail.

Flush and doorbell. `flush()` computes bytes = tail−buff, returns if 0, `__threadfence_system()`, writes the byte count to the mapped doorbell and spins until it reads 0, then resets tail, a fence, and head.

Host receive. `ChannelHost::recv` reads the doorbell, copies at most `max` bytes from the pinned buffer (default build, no `USE_ASYNC_STREAM`), and writes back the bytes left (0 releases the GPU). NIXNAN asks for one packet at a time.

Receiver thread. Loops while `recv_thread_started` and `recv_thread_receiving`; skips `to_skip()` packets; treats `warp()==-1` as end-of-kernel; otherwise formats "error [flags] detected in operand k of instruction S in function F [at path:line] of type T" through the mutex-guarded `nnout_line`.

Why two packets. The device pushes `skip=0` and `skip=1`. The receiver ignores the `skip=1` copy. The repository does not state why; my reading is that the second packet is a trailing marker that keeps a partially read buffer from being interpreted as a new event. *(my reading; unverified)*

The final sentinel message. The design: `flush_channel<<<1,1>>>` pushes `exception_info(int4{-1,-1,-1,-` and flushes, the receiver sees `warp()==-1`, clears `recv_thread_receiving`, and the exit callback waits for that flag with `sched_yield()`, so that no output of a kernel is lost and the next launch starts from a drained channel. **In commit `ce35185` the launch, the `skip_flag` manipulation and the wait loop are commented out in `nvbit_at_cuda_event`.** The receiver still handles the sentinel; nothing sends it. I did not test whether any report is lost because of this.

9 End-of-run report

`recorder->end()` copies the counters to the host. For each instruction, operand and flag value with non-zero count, counts are added to buckets keyed by (type, memory/not) and flag; "distinct" counts locations and "repeats" is total minus distinct. The report prints FP16, BF16, FP32, FP64 sections (and memory variants).

10 Part A: an ordinary SASS instruction

`rd_step_fp16` in `rd_nixnan` contains (from `cuobjdump -sass`) `HFMA2 R0, R0.H0_H0, c[0x0][0x170].H0_H0, R9.H0_H0`. The verbose log shows two injected calls: AFTER with 2 arguments after the fixed ones (one `reginfo`, one register: "operand 0, regs: 1, count: 2") and BEFORE with 6 (three `reginfo`, three registers: operands 1, 2, 3 with "regs: 1"). Operand 2 is the constant bank word. The report printed an infinity in operand 1 (source) and in operand 0 (destination) for this instruction, with function `rd_step_fp16` and type `f16`. No source line appears because the scalar binary has no line information.

11 Part B: HMMA

`HMMA.16816.F16 R26, R4, R30, R26` (SASS offset `0x840` in `rd_tc`). The JSON entry has four groups: D f16×4, A f16×8, B f16×4, C f16×4, i.e. 2, 4, 2, 2 registers. Observed in the log: AFTER "3 registers" (one `reginfo`+2) and BEFORE "11 registers" (three `reginfo`+8). Registers picked up for this instruction *(computed from the SASS operand text)*: D = R26,R27 (after); A = R4–R7, B = R30,R31, C = R26,R27 (before). Computed `reginfo` words, assuming GCC LSB-first bit-field allocation: 0x00010C04, 0x00022C08, 0x00014C04, 0x00016C04.

Observed reports (`fp16acc16`): infinity in operand 0 of HMMAs at step 230 (program's count: 120 inf), then NaN in operand 0 of HMMAs in the next step as inf operands meet zeros of P ($0\times\infty$), and all 4096 entries NaN by step 232. For `fp16acc32` and `bf16acc32` no `F2FP` line appears in the logs, and `instructions.json` contains no `F2FP` key (grep count 0), so the conversion that creates the first inf is not instrumented; the inf is first seen as an operand of the next HMMA. `LINE_INFO=1` did not yield `mma_sync` source locations in these runs.

12 Not covered

The histogram path (`fp-histogram.cu`), memory-instruction instrumentation (`meminstrumentation.cu`), and `TIME_KERNELS/SAMPLING` experiments were not studied here.